

Modélisation et simulation distribuée du mouvement d'un banc de poissons

Soutenance – Projet global

Yohann Front-Reignier Iram Madani-Fouatih

Master 1 CHPS – ISTY/UVSQ

Mai 2026

Plan

- 1 Contexte et objectifs
- 2 Méthode
- 3 Résultats
- 4 Cluster
- 5 Conclusion

- **Contexte applicatif** : systèmes multi-agents, bio-inspiration.
- **Problèmes** : coût de calcul qui explose avec le nombre de boids et manque de fiabilité avec le réel.
- **Objectif S2** : distribuer le calcul sans casser la cohérence du modèle.

- Simuler un banc de poissons via le modèle des Boids.
- Comparer les approches :
 - séquentielle,
 - parallélisme mémoire partagée (Kokkos/OpenMP).
- Mesurer les performances et la scalabilité.

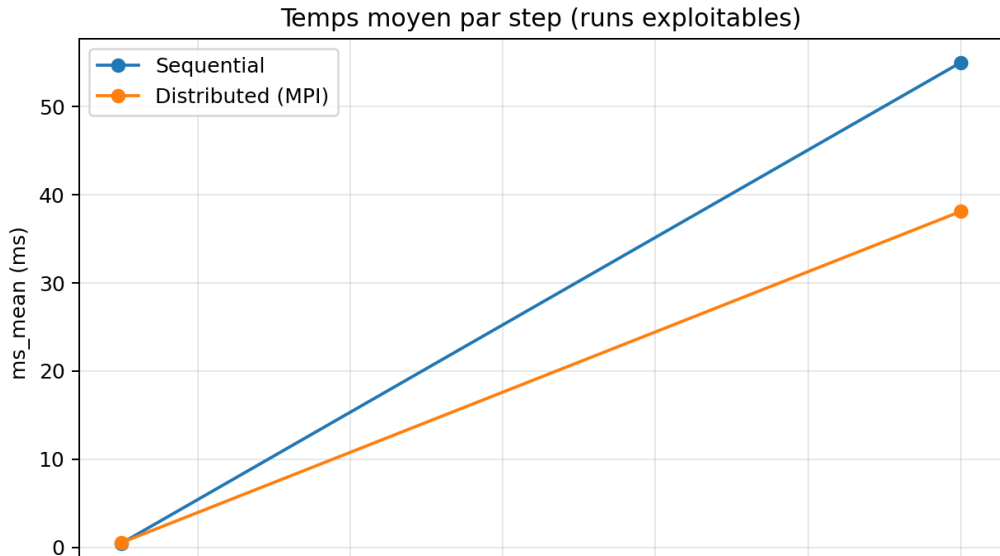
- Boid : comportement individuel (séparation, alignement, cohésion).
- Flock : gestion collective + grille spatiale.
- DistributedFlock : décomposition de domaine, halos, migration MPI.
- MPIManager : cohérence entre processus

Pipeline expérimental

- ① Build et validation locale.
- ② Exécution cluster (Slurm) : smoke, baseline, scaling, latence, multi-nud.
- ③ Collecte des logs et extraction CSV.
- ④ Analyse : temps moyen, speedup, stabilité.

- SpatialGrid vs naïf : jusqu'à **4.68x** de gain à $N = 800$.
- MPI strong scaling ($N = 5000$) :
 - 1 proc : 0.90x (overhead),
 - 2 procs : 1.91x,
 - 4 procs : **3.89x**.
- Run multi-nud exploitable ($N = 12000$) : **1.44x**.

Temps moyen par step (runs exploitables)



Speedup MPI (runs exploitables)

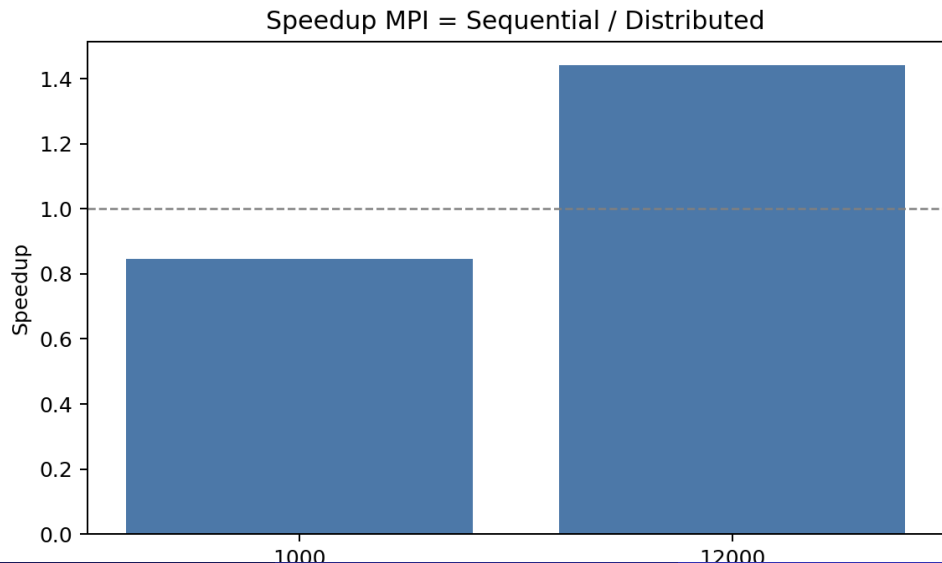


Tableau de résultats (à compléter)

Cas	Séquentiel	MPI	Speedup
N=1000			
N=5000			
N=12000			

- Gestionnaire Slurm (`sbatch`, `srun`).
- Modules utilisés : `gcc/13.2.0`, `openmpi/4.1.6`.
- Scripts dédiés : `smoke`, `baseline`, `scaling 1/2/4`, latence 10 min, multi-nud.
- Budget prévisionnel : **10.92 heures-curs** (quota 10000).

Limites et difficultés (à personnaliser)

- Runtime MPI (`libmpi_cxx.so`) et propagation d'environnement.
- Dépendances build (GTest/SFML) sur cluster.
- Temps limite Slurm sur certaines campagnes.

- Le modèle est fonctionnel et cohérent physiquement.
- La grille spatiale apporte le premier gain algorithmique majeur.
- MPI devient rentable quand la charge augmente.

- Augmenter le nombre de noeuds à + de 4
- Passage en calcul sur GPU
- Ajout d'erreur aléatoire sur le calcul des boids (voir les comportements avec des erreurs)
- Ajout d'obstacles ou de "prédateurs"
- Faire tourner sur cluster avec visualisation graphique (SFML non dispo sur Zen).

Merci